if গঠন এবং শর্তভিত্তিক পছন্দ

পাঠ 8 — 1IOT, SST-ICT-01 — যে প্রোগ্রাম ঠিক করে কী করতে হবে: if, if-else, নেস্টিং

29/09/2026

1 পুনরালোচনা: printf এবং scanf

টীকা

আগের পাঠের দ্রুত পুনরালোচনা। `printf` তৈরি করে যাকে বলে **আউটপুট**: আমাদের ক্ষেত্রে আউটপুট যায় মনিটরে। `scanf` দিয়ে **ইনপুট** পাওয়া যায়: আমাদের **ইনপুট** যন্ত্র হলো কিবোর্ড। প্রোগ্রামকে একটি **বাক্স** হিসেবে ভাবলে, আমরা বাক্স থেকে কিছু বের করে ব্যবহারকারীকে দেখাতে পারি, আবার বাক্সের ভেতরে তথ্য ঢোকাতেও পারি।

সতর্কতা

এই ধারণা (যে বাক্স **ইনপুট** নেয় ও আউটপুট দেয়) খুব গুরুত্বপূর্ণ, কারণ **Arduino** ব্যবহারের সময় এটি বারবার আসবে: Arduino এমন একটি বোর্ড, যাকে আমরা তথ্য পাঠাতে পারি এবং যার কাছ থেকে তথ্য নিতেও পারি, কী করছি তার ওপর নির্ভর করে।

সংজ্ঞা

লেখার নিয়মের সারসংক্ষেপ:

```
// ছাপায়; পূর্ণসংখ্যায় %d, float-এ %f
printf("testo %d", variabile);
// কিবোর্ড থেকে পড়ে; & কখনো ভুলো না
scanf("%d", &variabile);
```

`printf` ছাপা শেষে নিজে থেকে **কখনো** নতুন লাইনে যায় না: নতুন লাইনে যেতে হলে `\n` লাগে। `scanf` শুধু সংখ্যা নেয় (**কখনো** মুক্ত লেখা নয়, এটি কথোপকথনের কৃত্রিম বুদ্ধিমত্তা নয়): **পূর্ণসংখ্যার** জন্য `%d`, **দশমিকসহ** সংখ্যার জন্য `%f`।

উদাহরণ

বর্গক্ষেত্রের **ক্ষেত্রফলের** অনুশীলন, আগে করা, এই পাঠের শুরু হিসেবে আবার নেওয়া হলো:

```
int L;
printf("Inserisci il lato del quadrato: ");
scanf("%d", &L);

int area;
area = L * L;

printf("L'area del quadrato vale %d", area);
```

2 যন্ত্র কীভাবে চালায়: কাজের একটি ধারা

সংজ্ঞা

আগের উদাহরণের কোড যন্ত্র চালায় **কাজের একটি ক্রমবদ্ধ ধারা** হিসেবে, সবসময় একই ক্রমে:

1. **ভেরিয়েবল** `L` তৈরি করে (`int L;`)
2. বার্তা ছাপায় (`printf`)
3. **কিবোর্ড** থেকে ডেটা নেয় (`scanf`)
4. **ভেরিয়েবল** `area` তৈরি করে (`int area;`)
5. `L * L` হিসাব করে
6. ফলাফল ছাপায় (`printf`)

আসলে প্রোগ্রামের সংজ্ঞাই হলো **কাজের একটি ক্রমবদ্ধ তালিকা**: 42 বার চালালেও সবসময় শুধু এই একই ধারা করবে, **কখনো** অন্য কিছু নয়।

সতর্কতা

সমস্যা: যে প্রোগ্রাম সবসময় একই কাজ করে তা কাজের নয়। ক্লাসে ব্যবহৃত তুলনা: একটি ভিডিও গেম, যেখানে একমাত্র কাজ সবসময় একই ("শুরু করতে এখানে চাপো", খেলা শেষ), মজার বা কাজের হবে না। এমন প্রোগ্রাম দরকার যা পরিস্থিতি অনুযায়ী **কী করবে বেছে নিতে** পারে।

3 ফ্লোচার্ট: ধারা বনাম পছন্দ

টীকা

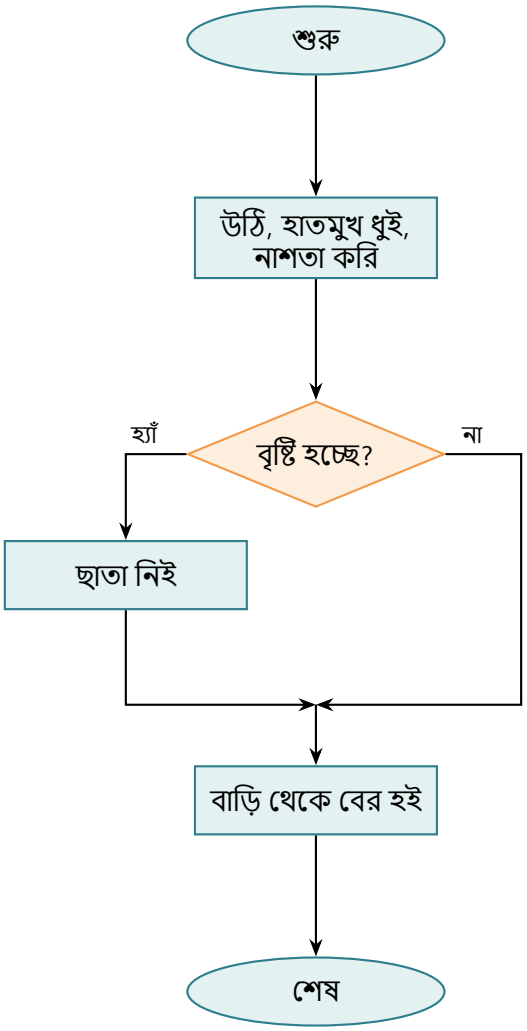
ক্লাসে ব্যবহৃত চিত্ররূপ: একটি **আয়তক্ষেত্র** একটি কাজ বোঝায় (যেমন **ভেরিয়েবল** তৈরি, বার্তা ছাপানো)। তীর দিয়ে জোড়া আয়তক্ষেত্রের একটি **ধারা** কাজের সরল প্রবাহ বোঝায়, ভিডিও গেমের স্তরের মতো: প্রথম, দ্বিতীয়, তৃতীয়... সবসময় একই ক্রমে।

সংজ্ঞা

প্রবাহে একটি **পছন্দ** দেখাতে একটি নতুন প্রতীক লাগে, **রম্বস** (**rombo**): এর ভেতরে এমন একটি প্রশ্ন লেখা হয় যার উত্তর **সত্য**/মিথ্যা (হ্যাঁ/না)। রম্বস থেকে দুটি তীর বের হয়, প্রতিটি সম্ভাব্য উত্তরের জন্য একটি, যা প্রোগ্রামের আলাদা পথে নিয়ে যায়।

উদাহরণ

ধারণাটি শুরু করতে ব্যবহৃত **দৈনন্দিন** **জীবনের** **তুলনা**: বৃষ্টি হোক বা রোদ, সবসময় ছাতা নেওয়া ভুল নয়, কিন্তু ঝামেলার। এর চেয়ে ভালো: "আমি উঠি, হাতমুখ ধুই, পোশাক পরি, নাশতা করি (সবসময়ই)। বাইরে তাকাই: বৃষ্টি হলে ছাতা নিই; না হলে বাড়িতে রেখে যাই।" একে বলে **সিদ্ধান্তের গঠন** (**costrutto decisionale**): এটি **পছন্দ** করতে দেয়, অর্থাৎ একটি **শর্ত** (**condizione**) অনুযায়ী প্রবাহকে এক বা অন্য পথে পাঠায়।



ছবি/নকশা 1: ক্লাসে আঁকা ফ্লোচার্ট: "বৃষ্টি হচ্ছে?" রম্বস প্রবাহকে দুটি আলাদা পথে ভাগ করে, যেগুলো এগোনোর আগে আবার মিলে যায়।

4 if গঠন

সংজ্ঞা

`if` (ইংরেজিতে "যদি") হলো সেই **গঠন** (**costrutto**), যা **ফ্লোচার্টের** রম্বসকে কোডে রূপান্তর করে:

```
if (condizione)
{
    // শর্ত সত্য হলেই শুধু এই কোড চলে
}
```

দুটি **কার্লি** **বন্ধনী** বাধ্যতামূলক: এগুলো একটি আধার তৈরি করে, যা ঠিক করে দেয় কোন নির্দেশগুলো "**সত্য** হলে চালাও" **ব্লকের** অংশ আর কোনগুলো কোডের স্বাভাবিক ধারাবাহিকতা। **বন্ধনী** ছাড়া যন্ত্র এই দুটি আলাদা করতে পারত না।

টীকা

`if` C লাইব্রেরির কোনো সাধারণ **ফাংশন** নয়: এটি একটি **ভাষার** **গঠন**, অর্থাৎ একটি মূল শব্দ যা প্রায় সব লেখাভিত্তিক প্রোগ্রামিং ভাষায় আছে (Python, C#, Delphi/Object Pascal...): শর্তভিত্তিক **পছন্দ** এত মৌলিক প্রয়োজন যে এটি না থাকা খুবই বিরল ব্যতিক্রম। PLC নিয়ে শিক্ষক আগেই জানিয়েছেন, সেগুলো পড়ার সময় একই ধরনের **শর্ত** প্রকাশের উপায় পাওয়া যাবে: PLC-র সবচেয়ে প্রচলিত কন্টাক্ট ভাষায় (**ladder** **logic**) আসল `if` শব্দ নেই – একই শর্তের যুক্তি পাওয়া যায় মইয়ের মতো নকশায় কন্টাক্ট ও কয়েল সাজিয়ে; **Structured** **Text**-এর মতো নতুন PLC ভাষায় আবার C-র মতোই লেখাভিত্তিক `if` আছে।

সংজ্ঞা

বন্ধনীর ভেতরের **শর্ত** তৈরি হয় আগের পাঠে দেখা **শর্তের** **অপারেটর** দিয়ে (যেমন `>`, `>=`, `<`, `==`): এই প্রকাশগুলোর মান সবসময় সত্য বা মিথ্যা, **কখনো** মাঝামাঝি নয়।

5 নির্দেশিত উদাহরণ: ছাত্রের নম্বর

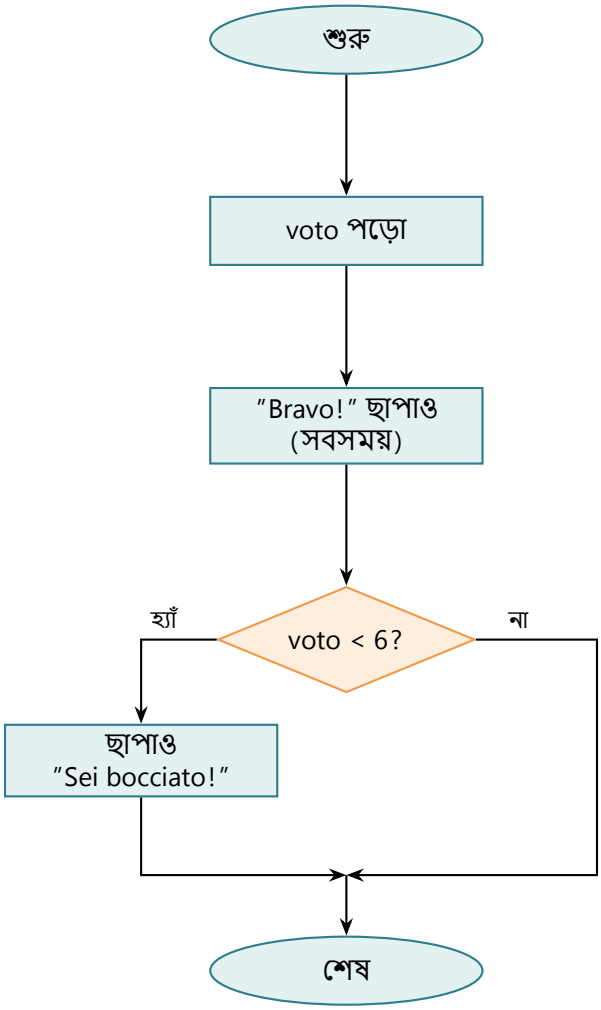
উদাহরণ

ক্লাসে সরাসরি করা উদাহরণ, একটি অনলাইন কম্পাইলারে আসলেই চালিয়ে: একটি প্রোগ্রাম যা একজন ছাত্রের নম্বর (**voto**, দশের মধ্যে, **পূর্ণসংখ্যা**) চায়। সরাসরি লেখা প্রথম সংস্করণ, যেখানে ইতিবাচক বার্তা সবসময় ছাপা হয়, আর কম নম্বর হলে শুধু `if` দিয়ে আরেকটি বার্তা যোগ হয়:

```
int voto;
printf("Inserisci il voto dell'allievo: ");
scanf("%d", &voto);

printf("Bravo!");

if (voto < 6)
{
    printf("Sei bocciato!");
}
```



ছবি/নকশা 2: প্রথম সংস্করণের ফ্লোচার্ট (bug সহ): "Bravo!" ছাপা হয় সবসময়, if-এর শাখা যাই হোক, কারণ এটি শর্তের ব্লকের বাইরে।

সতর্কতা

ক্লাসে একসাথে পাওয়া bug (**Bravo** = সাবাশ, **Sei bocciato** = তুমি ফেল): 8 নম্বরে প্রোগ্রাম ঠিকভাবে শুধু "Bravo!" ছাপায়। কিন্তু 3 নম্বরে ছাপায় **"Bravo!Sei bocciato!"**, অর্থাৎ **দুটি** বার্তাই: প্রথম `printf("Bravo!")` যাই হোক চলে, কারণ এটি `if`-এর বাইরে, তাই এটি **ধারার** একটি সাধারণ নির্দেশ, কোনো **শর্তের** সাথে যুক্ত নয়। `if` একা শুধু **শর্ত** সত্য হলে একটি নির্দেশ **যোগ করতে** পারে: বাইরে আগে লেখা নির্দেশকে **বদলাতে** বা **আটকাতে** পারে না। **নাহলে** (অর্থাৎ "Bravo!" ছাপানোর বদলে) কী করতে হবে তা স্পষ্ট করে বলার একটি উপায় নেই।

6 if-else রূপ

সংজ্ঞা

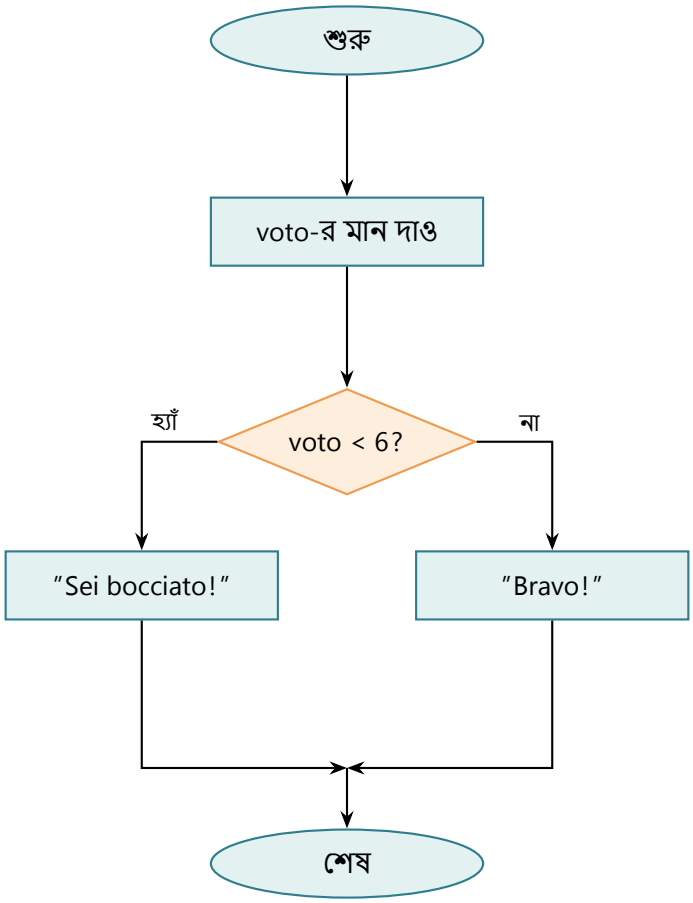
`if…else` "নাহলে" **শাখা** (**ramo**) যোগ করে: শর্ত **সত্য** হলে প্রথম ব্লক চলে, **মিথ্যা** হলে `else` **ব্লক** চলে। দুটি ব্লক সবসময় পরস্পরকে বাদ দেয়: দুটির একটিই চলে।

```c
int voto;
printf("Inserisci il voto dell'allievo: ");
scanf("%d", &voto);

if (voto < 6)
{
    printf("Sei bocciato!");
}
else
{
    printf("Bravo!");
}
```

উদাহরণ

ওপরের প্রোগ্রামে ক্লাসে সরাসরি করা পরীক্ষা: 8 নম্বরে প্রোগ্রাম ছাপায় "Bravo!"; 3 নম্বরে ছাপায় "Sei bocciato!"। ব্যবহারকারীর দেওয়া ডেটা অনুযায়ী প্রোগ্রামের আচরণ সত্যিই বদলায়: এটি আর স্থির **ধারা** নয়।



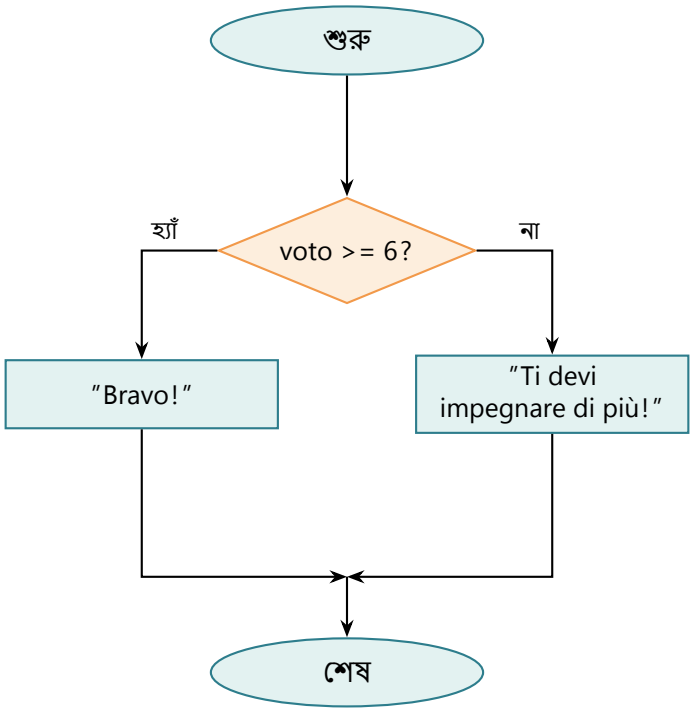
ছবি/নকশা 3: `if-else`-এর ফ্লোচার্ট: দুটি শাখা পরস্পরকে বাদ দেয় (সবসময় দুটি ব্লকের একটিই চলে), তারপর এগোনোর আগে প্রবাহ আবার মিলে যায়।

টীকা

একই সমস্যা "উল্টো করেও" লেখা যায়, শর্ত ও বার্তা **অদলবদল** করে, ফলাফল একই:

```
if (voto >= 6)
{
    printf("Bravo!");
}
else
{
    // ফেল
    printf("Ti devi impegnare di piu'!");
}
```

দশমিকসহ নম্বর নিতে **ফরম্যাট নির্দেশক** বদলে `scanf("%f", &voto);` করলেই হয়, আর `voto`-কে `float` ঘোষণা করো: **`if-else`**-এর বাকি যুক্তি বদলায় না।



ছবি/নকশা 4: "উল্টো" সংস্করণের ফ্লোচার্ট: আগের if-else-এর একই যুক্তি, শর্ত ও বার্তা অদলবদল, ফলাফল একই (**Ti devi impegnare di più** = তোমাকে আরও চেষ্টা করতে হবে)।

7 সীমানায় সাবধান: বড় বনাম বড় বা সমান

সতর্কতা

খুব সাধারণ ভুল: 6-কেও পাসের নম্বর ধরতে চাইলেও `if (voto > 6)` লেখা। যন্ত্র **কঠোর**, উদ্দেশ্য বোঝে না: `voto` ঠিক `6` হলে `voto > 6` **শর্ত** **মিথ্যা** (6, 6-এর চেয়ে বড় নয়), তাই এভাবে লেখা `if`-এ 6 পাওয়া ছাত্র "ফেল" হবে। সীমার মানটিও ধরতে **বড়** **বা** **সমান** অপারেটর লাগে: `voto >= 6`। এসব ক্ষেত্রে **ভুল** সবসময় প্রোগ্রামারের, যন্ত্রের নয়: **শর্ত** সঠিকভাবে লিখতে হবে।

8 নেস্টিং: দুইয়ের বেশি সম্ভাবনা

সংজ্ঞা

আলাদা করার সম্ভাবনা দুইয়ের বেশি হলে (শুধু "ফেল/পাস" নয়, যেমন মূল্যায়নের চারটি স্তর), **নেস্টিং** (**annidamento**) ব্যবহার হয়: একটি `if`-এর ভেতরে আরেকটি `if`, যত স্তর দরকার। প্রতিটি **নেস্টেড** `if`-এর নিজের দুটি বাধ্যতামূলক **বন্ধনী** থাকে।

উদাহরণ

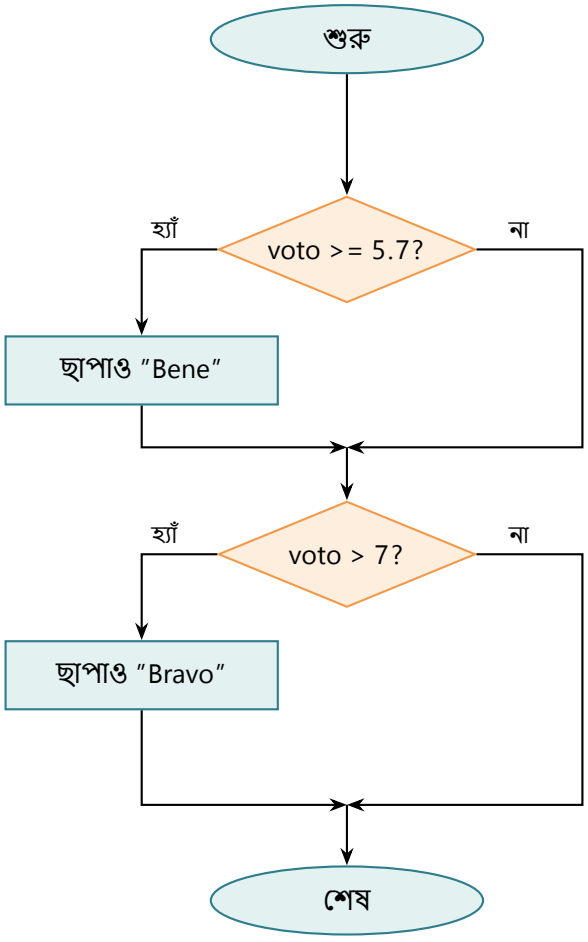
ক্লাসে আলোচিত লক্ষ্য: নম্বরের ওপর মূল্যায়নের স্তর তৈরি করা — 5-এর নিচে "ফেল", 5 থেকে 6 "আরও চেষ্টা করতে হবে", 6 থেকে 7 "সাবাশ", 7-এর ওপরে "প্রতিভা"। প্রতিটি স্তরের নিজস্ব **শর্ত** লাগে, স্তরগুলো একে অপরকে কীভাবে সীমিত করে তা খেয়াল রেখে।

সতর্কতা

এড়ানোর মতো একটি ক্লাসিক ভুল: **শর্তগুলো** পরস্পরকে বাদ না দিলেও **নেস্টেড** না করে দুটি **পরপর** `if` লেখা। ক্লাসে আলোচিত উদাহরণ:

```
// নেস্টেড নয়: আলাদা if
if (voto >= 5.7)
{
    printf("Bene");
}
// নেস্টেড নয়: আলাদা if
if (voto > 7)
{
    printf("Bravo");
}
```

৪ নম্বরে এই কোড "Bene" **এবং** "Bravo" **দুটিই** ছাপায়, কারণ ৪ দুটি স্বাধীন শর্তই পূরণ করে: প্রতি নম্বরে একটি বার্তা চাইলে এটি কাঙ্ক্ষিত আচরণ নয়।



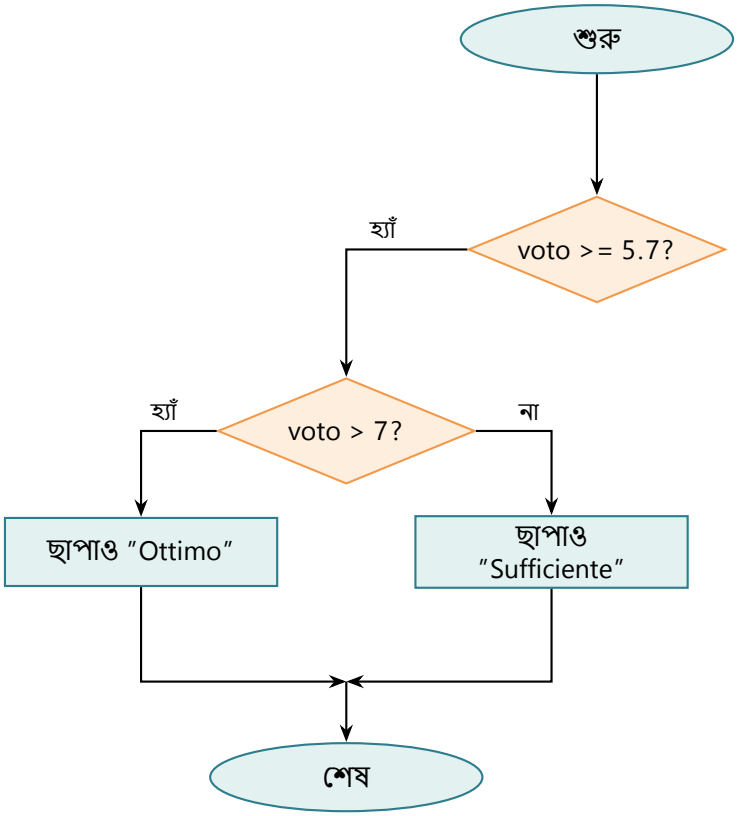
ছবি/নকশা 5: ভুলটির ফ্লোচার্ট: দুটি পরপর `if` (নেস্টেড নয়) প্রথমটির ফলাফল যাই হোক, সবসময় দুটিই যাচাই হয় – ৪ নম্বরে "Bene" আর "Bravo" দুটিই ছাপা হয়।

সংজ্ঞা

নেস্টিং দিয়ে সঠিক সমাধান: দ্বিতীয় `if` প্রথমটির ব্লকের ভেতরে লিখতে হবে, যাতে প্রথম শর্ত **সত্য** বলে নিশ্চিত হওয়ার **পরেই** এটি যাচাই হয়:

```
if (voto >= 5.7)
{
    if (voto > 7)
    {
        printf("Ottimo");
    }
    else
    {
        printf("Sufficiente");
    }
}
```

এই গঠনে: `voto` `5.7`, `6` বা `6.9` হলে প্রোগ্রাম ছাপায় "Sufficiente" (প্রথম **`if`**-এ ঢুকেছে, দ্বিতীয়টিতে নয়); `7.1` হলে ছাপায় "Ottimo" (দুটি **শর্তই** পূরণ করে); `5.7`-এর নিচে হলে **ব্লকে** ঢোকেই না (এই সরল উদাহরণে সেই ক্ষেত্রে কোনো বার্তা ছাপা হয় না; এটি বাইরের একটি **`else`** দিয়ে সামলাতে হতো)।



ছবি/নকশা 6: নেস্টিংসহ সঠিক সমাধানের ফ্লোচার্ট: দ্বিতীয় `if` (voto > 7?) যাচাই হয় শুধু প্রথমটি সত্য হলে – তাই একসাথে দুটি বার্তা কখনো ছাপা হয় না। প্রথম if-এর "না" শাখা (5.7-এর নিচে নম্বর) দেখানো হয়নি: এই সরল উদাহরণে সেই ক্ষেত্রে কোনো বার্তা ছাপা হয় না।

9 এই পাঠ থেকে যা মনে রাখতে হবে

- **পছন্দ** ছাড়া প্রোগ্রাম প্রতিবার একই কাজের **ধারা** চালায়: এটি কাজের নয়।
- **ফ্লোচার্ট** পছন্দ দেখায় একটি **রম্বস** (সত্য/মিথ্যা প্রশ্ন) আর দুটি বের হওয়া তীর দিয়ে।
- `if (condizione) { …}` শর্ত সত্য হলেই **শুধু** ব্লকটি চালায়; **বন্ধনী** সবসময় বাধ্যতামূলক।
- `if …else …` "নাহলে" **শাখা** যোগ করে: দুটি ব্লক সবসময় পরস্পরকে বাদ দেয়।
- **সীমানায়** সাবধান: `>` সীমার মান বাদ দেয়, `>=` তা ধরে। যন্ত্র **শর্ত** ঠিক যেমন লেখা তেমন প্রয়োগ করে, উদ্দেশ্য বোঝে না।
- দুইয়ের বেশি সম্ভাবনায় **নেস্টিং** (`if`-এর ভেতরে `if`) ব্যবহার হয়: শর্ত মিলে যাওয়া দুটি পরপর, **নেস্টেড** নয় এমন `if` **ভুলবশত** একসাথে একাধিক ব্লক চালাতে পারে।
- কোডের পাঠযোগ্যতা নষ্ট না করতে **নেস্টিং** সীমিত রাখতে হবে।

উদাহরণ

পরের পাঠের জন্য করণীয়: বহু স্তরের **নেস্টিং** ধরার আগে, ক্লাসে **`if`** আর `if-else` নিয়ে অনুশীলন করা, সংখ্যার উদাহরণে (নম্বর, মাপ) **শর্ত** তৈরি করে।

10 শব্দকোষ: ক্লাসে ব্যবহৃত ইতালীয় শব্দ

- **voto**: নম্বর
- **condizione**: শর্ত
- **vero / falso**: সত্য / মিথ্যা
- **sì / no**: হ্যাঁ / না
- **altrimenti**: নাহলে (`else`)
- **ramo / blocco**: শাখা / ব্লক
- **annidamento**: নেস্টিং
- **maggiore / minore**: বড় / ছোট
- **Bene, Bravo, Ottimo**: ভালো, সাবাশ, চমৎকার
- **Sufficiente**: পাসের মতো
- **bocciato / promosso**: ফেল / পাস